

Software / ML Domain Taskphase

Conceptual Report — Task 6

Participant Name: Abhigyan

Roll Number / ID:

240961320

Domain: Software / ML

GitHub Repository: https://github.com/pandeyabhigyan005-hue/Synergy_TP

Submission Date: [28/06/2026]

Abstract

This report covers the concepts behind Tasks 1 through 5 of the Software/ML Domain Taskphase. The work moved through five stages: setting up a development environment, writing structured Python programs, reading CSV data two different ways, cleaning a dataset that was genuinely messy, and turning the cleaned data into charts.

1. Introduction

The purpose of the taskphase work until now was to develop fundamental data handling and processing skills. The tasks introduced concepts such as version control using Git and GitHub, Python programming, CSV parsing, data cleaning with pandas, and data visualization using matplotlib.

2. Development Environment and Version Control

2.1 Git and GitHub

Git keeps a timestamped record of every change made to a project. The alternative, saving files as `report_v1.py` and `report_final2.py` and `report_ACTUALFINAL.py`, works until it doesn't. With Git, every saved state comes with a message explaining what changed, and going back to any earlier state takes one command.

GitHub puts that history online so the project can be cloned, reviewed, or run by anyone with the link. Clean repository hygiene means keeping the code repository organized, clean (free of unnecessary files or mistakes) and easy to understand.

2.2 Python Virtual Environments

A Python virtual environment gives each project its own copy of Python or in simple terms is a python installation created specifically for a project .It allows different projects on the same computer to use different versions of packages without any conflict.

The `requirements.txt` file is an archive which contains the packages the environment uses and at what versions. Anyone who wants to run the project on a different machine runs `pip install -r requirements.txt` and gets an identical setup. This is the simplest version of reproducibility, and it matters more than it sounds.

2.3 The .gitignore File

The `.gitignore` file tells Git which files and folders it should not track. Virtual environment folders and `py_cache` directories serve no purpose in the repository, `.gitignore` keeps those things out from the start.

2.4 Linux Command-Line Basics

The terminal is really the way to get around a file system and look at files. When you use the command `pwd` it shows you where you are. If you use `ls` it shows you what is there. You can use `mkdir`, `Cp` and `mv` and `rm` to do the basic things with files. These things are what you need to do every day. The commands `grep` and `find` are really helpful. At first they might not seem like much. They are. `Grep` looks inside files for text. `Find` helps you locate files by their name

or what type of file they are. You can use `chmod` to change the permissions on a file. The command `wc` counts how many lines are, in a file. These are not things to learn but if you know how to use them you can look at a project or fix a script without even opening a file manager.

3. Python and File Handling Concepts

3.1 Functions and Type Hints

A program that does everything in one place is hard to debug and harder to reuse. Breaking it into functions, each doing one job, means a bug can be isolated to the specific function causing it. It also means that function can be tested on its own and used again elsewhere.

Type hints make functions easier to read. Writing `def read_file(path: str) -> list[dict]` tells anyone reading the code exactly what goes in and what comes out, without needing to trace through the body. They do not affect how Python runs the code, but they reduce confusion significantly when someone (including your future self) reads it later.

3.2 File I/O

File handling enables programs to read data from external files and write processed results back to disk.. Files left open accidentally can cause data loss or errors in longer-running programs.

The taskphase involved reading CSV files, writing JSON summaries, saving cleaned datasets, generating markdown reports, and exporting visualization summaries.

3.3 Exception Handling

Exception handling prevents programs from terminating unexpectedly when invalid or missing data is encountered. Without exception handling, the program crashes on the first problem it encounters. With `try/except`, it can catch the problem, report it, and keep going. That difference matters a lot when you are processing hundreds of rows and one bad one should not stop everything.

4. CSV Parsing and Pandas

4.1 CSV as a Data Format

A CSV file is plain text. Each line is a row. Values within a line are separated by commas. The first line names the columns.

4.2 Manual CSV Parsing

In Task 3, a CSV parser was built from scratch using only `open()`, line iteration, and `str.split()`. Each line became a dictionary with the header names as keys and the row values as values. Type conversion and blank-line skipping had to be written explicitly.

This was slower to write than using a library, and it showed exactly why libraries exist. Quoted commas, multi-line fields, and encoding edge cases all require extra code that adds up fast.

4.3 Pandas-Based Loading

`pd.read_csv()` reads the header, parses the rows, and returns a DataFrame, which is a table that supports column operations, filtering and grouping. Pandas also has built-in methods for dealing with missing values and wrong data types.

Running both approaches on the same file in Task 3 produced the same numbers. The gap between them shows up with messier data or larger files, where writing the edge cases by hand takes longer than just using the library.

5. Data Cleaning

5.1 Why Data is Messy

Data from real sources is rarely clean. A form lets people enter whatever they want. A spreadsheet gets edited by multiple people over time. Someone types "ML" where someone else typed "ml" and a third person typed "MACHINE LEARNING." Someone leaves a field blank. Someone accidentally submits the same row twice. None of this is unusual.

Cleaning is not an optional step before the "real" work. It is the work. A model or analysis built on dirty data produces wrong results, and the errors are often hard to trace back to the source.

5.2 Handling Duplicates

Duplicate rows skew counts and averages. In Task 4, student S005 appeared twice with identical values. Removing exact duplicates is straightforward, but it requires a decision about what counts as a duplicate first. Two rows for the same student in different months are not duplicates. Two rows for the same student with identical data are.

5.3 Standardizing Categories

When the same category appears as "ml", "ML", and "MACHINE LEARNING" in the same column, any analysis that groups by that column will treat them as three separate groups. The fix is a lookup dictionary that maps all variations to one canonical value. New variations get added to the dictionary as they appear.

5.4 Type Conversion and Unit Normalization

CSV files store everything as text, so type conversion is always required before any arithmetic. Percentages might come in as "92%" with the symbol attached. Heights might be "170 cm" in some rows and "1.62 m" in others. Scores might be "nine" spelled out in words. Each of these needs different handling: strip the unit, detect the format, convert to a standard representation.

Word-based numbers need their own mapping from string to integer.

5.5 Handling Missing Values

A blank field is not a zero. A missing score usually means the value was never entered, not that the student scored nothing. The right strategy depends on the column: numeric fields can be filled with the column median, but that decision has to be made deliberately and documented. Blindly dropping every row with a missing value can remove more data than you can afford to lose.

5.6 Validation After Cleaning

After cleaning, the data needs to be checked. Validation confirms that the problems were actually fixed: no remaining duplicates, attendance values between 0 and 100, only the expected category labels, no empty critical columns. Without this check, it is easy to think the data is clean when a quiet bug left something broken. Validation is what makes the cleaning trustworthy.

6. Data Visualization

6.1 Purpose of Visualization

A table of numbers requires reading to interpret. A chart makes the same information visible in seconds. That is not a small difference.

6.2 The Three Generated Plots `domain_average_score.png` shows average score by domain as a bar chart. It shows which group did better. Each bar is one domain; the height is the average score.

`attendance_vs_score.png` is a scatter plot with attendance percentage on the x-axis and score on the y-axis. Each point is one student. The plot shows whether students who show up more tend to score higher.

`submission_status_count.png` has two bars: submitted and not submitted. It shows participation at a glance.

7. Summary Tables and Workflow Diagram

Table 1: Tasks and Key Concepts

Task	Main Tool / Method	Key Concept Learned
Task 1	Github, venv, Linux WSL	Repository hygiene, isolated environments, and navigating the terminal.

Task 2	Python fundamentals	Functions, type hints, file I/O, and exception handling.
Task 3	Manual parser + pandas	How CSV text becomes structured data; why libraries handle edge cases that manual parsing cannot.
Task 4	pandas data cleaning	Identifying and fixing duplicates, type errors, missing values, and inconsistent categories in real data.
Task 5	matplotlib	Producing labeled, saved charts from cleaned data.

Table 1 — Summary of Tasks 1–5 and the main concept learned from each task.

Figure 1: Data Pipeline — From Raw CSV to Visualized Insights

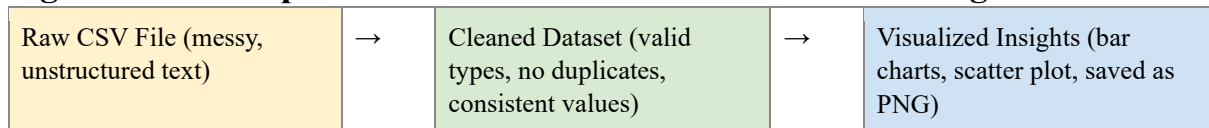


Figure 1 — Workflow from raw CSV through cleaning to final plots.

8. Conclusion

The five tasks form a complete pipeline, from an empty folder to a set of charts. Task 1 put the environment in order. Task 2 built the Python patterns that appear in almost every data script. Task 3 showed what CSV parsing actually involves underneath the library. Task 4 showed that data cleaning is harder than it looks and requires deliberate decisions, not just running a function. Task 5 put the cleaned data into a form that is actually readable.

The deeper point is that each step has to be done properly for the next one to work. A badly structured repository causes problems later. Skipping type conversion breaks the analysis. Skipping validation means errors get carried forward silently. None of the individual steps are complicated.

References

1. Python Software Foundation. Python Documentation — The Python Standard Library. <https://docs.python.org/3/library/>
2. pandas Development Team. pandas Documentation — User Guide. https://pandas.pydata.org/docs/user_guide/

3. Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in Science and Engineering, 9(3), 90-95.
4. Chacon, S. and Straub, B. Pro Git (2nd ed.). Apress. <https://git-scm.com/book/en/v2>
5. Python Packaging Authority. pip and virtual environments. <https://packaging.python.org/en/latest/guides/installing-using-pip-and-virtual-environments/>
6. The Linux Documentation Project. GNU/Linux Command-Line Tools Summary. <https://tldp.org/>